

这篇文章想解决的就是“云上共享数据时，用户撤销（revocation）太麻烦/太慢”的问题，尤其是基于属性的访问控制（ABE/attribute revocation）那套方案里，撤销往往牵一发而动全身。

传统较常见的做法是给属性/密文/密钥都塞一个版本号：撤销某个用户对某个密文的权限时，要更新该密文的版本信息，还要让其他没被撤销的用户更新对应的密钥；撤销某个用户对所有密文的权限时，相关密文要批量更新，其他用户也要更新密钥，开销还会跟“被撤销用户关联了多少属性”线性增长。

作者认为这会严重影响撤销的“有效性/效率”，尤其在属性多、用户多、设备算力弱（移动端/IoT）的场景里更明显。

他们的核心目标是做一个“灵活撤销”：既能撤销某个用户对某个指定密文（某个文件）的访问，也能一键撤销该用户对所有可解密密文（所有文件）的访问，并且尽量少做更新操作。

为了解决“撤销指定文件时不要更新密文、不要让其他用户更新密钥”，他们引入了一个边缘服务器（edge server）来帮忙做计算外包，并设计了一个类似“钥匙打孔/密钥穿孔（key puncturing）”的机制：需要撤销时，让边缘侧把变换密钥（transformation key）在某些标签（tag）上打孔，从而让被打孔的密钥无法再解对应标签的密文。

这样做的直观效果是：撤销某个用户对某个文件的访问时，不需要重新加密/更新该文件密文，也不用影响其他用户的密钥（不用大家一起升级版本、也不用维护多套版本密钥）。

对于“撤销某个用户对所有文件的访问”这类更重的操作，文章用到“密钥拆分（key splitting）”：把用户密钥拆成两类--跟属性有关的部分由授权机构（authority）发，跟周期/时段（period）有关的授权部分由数据拥有者（data owner）来分发。

我理解这个拆分的用意是把权限控制权拆开：当需要做全局撤销或更新时，非撤销用户的更新负担不再跟属性数量绑定，从而把非撤销用户的更新开销压到常数级（至少不再线性爆炸）。

文章里还提到安全性方面：给出了在其安全模型下的安全证明，强调撤销后应满足前向安全（forward security），也就是被撤销者不能继续访问之前能访问的密文。

性能结果里有个挺直观的数据：在资源受限设备上，属性数到 50 时，非撤销用户更新密钥大约只要 0.02 ms，更新速度提升约 25 倍（相对传统需要大量更新的方案）。

整体读下来，这篇更像是在“系统/机制层面”把撤销的工作从‘更新密文+全体用户联动升级’改成‘边缘侧打孔+权限拆分’，目的就是把撤销从大规模维护动作变成低成本动作，适合云-边-端协同的共享场景。